RoleReady PRD (MVP)

Team 44 · Outskill AI Accelerator hackathon · v0.1, 29 Oct 2026 Live doc: https://claude.ai/artifact/EdKkDmUh6TgUEcswb2hSZE · Architecture: https://claude.ai/artifact/DWSb3SnLwHh4m4qZoN4Ysj Put this file in the repo at `docs/PRD.md` so the coding agent can read it.

1. Overview

RoleReady is an AI interview coach that turns a job seeker's resume and a target job description into 6 tailored mock-interview questions, then scores each typed answer with feedback grounded in the candidate's own resume.

Problem. Job seekers with an interview days away can't get realistic, honest practice for that specific role. Question lists are generic, human mock interviews are costly and hard to book, and current AI tools grade delivery or generic answers rather than whether the candidate used the right evidence from their own resume for this job. So they walk in rehearsed on the wrong things.

Target user. Early- to mid-career job seeker (0–8 years, incl. career switchers) with an interview in the next 1–14 days, practising on a laptop at home.

Top 3 pains. (1) Generic practice that ignores their resume and this JD. (2) No honest feedback; they can't tell which project to talk about. (3) Freezing when speaking (voice should-have, not this MVP).

Goal. A live product where a new user goes from sign-in to first scored feedback in under 3 minutes, with 6/6 questions visibly tied to a resume line or JD requirement.

Differentiator. Every question and every piece of feedback cites the resume line (R1..Rn) or JD requirement (J1..Jn) it rests on, and claims the resume doesn't back are flagged.

2. Scope

Feature	Priority	Notes
Sign in (email + password)	Must	Supabase Auth, "confirm email" off
Resume PDF upload + text extraction	Must	Private bucket, PDF ≤ 5 MB
Paste job description	Must	Plus optional role title, company
6 tailored questions with "why we're asking" ⭐	Must	Each cites ≥ 1 R or J id
Text answers	Must	20–4,000 characters
Scored feedback with evidence check ⭐	Must	4 criteria, overall 0–100, what worked, one fix, stronger answer, unsupported claims
Saved session + History	Must	Reopen read-only
Voice mock (Retell) ⭐	Should	Fallback: browser speech-to-text
Email report (n8n + Gmail) ⭐	Should	On session complete

Out of scope: company-research agent, Pinecone bank, HeyGen/ElevenLabs, Calendar, JD coverage map, progress charts, delivery/video analytics, live copilot, resume builder, payments, teams, multi-language, mobile app, social sign-in, password reset UI.

3. User flow and screens

Two flows to build first: **create a session** (1–3) and **answer and get feedback** (4–5).

1. **Sign in** `/login`: email + password with “create account” toggle. Signed-out users redirect here.
2. **New session** `/new`: upload PDF, paste JD, optional role title and company, “Generate my interview”.
3. **Preparing** (progress state on `/new`): “Reading your resume” → “Writing your questions” (10–30 s).
4. **Interview** `/session/:id`: 6 questions; each shows type, “why we’re asking”, and R/J chips whose hover shows the cited text. One answer box each with counter and “Get feedback”.
5. **Feedback panel** (inline): overall 0–100, four 1–5 bars (relevance, evidence, structure, impact), what worked, one fix, stronger answer with copy button, unsupported claims as warnings. “Try again” overwrites.
6. **Session summary** (top of `/session/:id` when all 6 answered): overall + per-question scores.
7. **History** `/history` (home after sign-in): sessions newest first with role, company, date, status, score.

4. Requirements and acceptance criteria (checked on the Vercel URL, mocks off)

ID	Requirement	Acceptance check
F1	Sign up / in / out	New account reaches History; refresh keeps session; signed-out `/session/:id` → `/login`
F2	Create session from PDF + JD	Non-PDF or > 5 MB rejected; JD < 200 chars rejected; PDF at `resumes/{user_id}/{session_id}.pdf`
F3	Parse resume + JD	R1..Rn lines and 5–10 J requirements saved; image-only PDF → `parse_failed` message
F4	6 grounded questions	Exactly 6, mixed types, each cites a real R/J id; chips hoverable
F5	Text answer	Button disabled < 20 or > 4,000 chars; loading state
F6	Score	4 scores 1–5; overall = round((sum − 4) / 16 × 100); what worked; one fix; stronger answer from resume facts only; unsupported claims
F7	Re-answer	Overwrites previous feedback
F8	Complete	After 6th answer status `completed`; overall on summary and History
F9	History + reopen	Own sessions only, newest first; reopen makes no LLM calls
F10	Errors	Any ApiError → friendly message + retry; never a blank page

Non-functional: questions < 30 s, each score < 15 s; LLM keys only in Supabase secrets; RLS on every table and the bucket; demo with sample resumes (free Gemini tier may train on prompts); LLM JSON validated with 1 retry; responsive to 375 px, keyboard usable; free tiers only.

5. Data, contracts and stack

Source of truth: `supabase/functions/_shared/contracts.ts` and `supabase/migrations/0001_init.sql` (shared folder `phase2-architecture/starter/`).

Table	Key fields	Written by
`sessions`	role_title, company, jd_text, resume_path, resume_lines, jd_requirements, status (draft → ready → in_progress → completed), overall_score	App, parse-resume, generate-questions, score-answer
`questions`	session_id, position 1–6, text, type, why_asking, resume_refs, jd_refs	generate-questions
`answers`	question_id (unique), answer_text, scores, overall, what_worked, one_fix, stronger_answer, unsupported_claims	score-answer
Storage `resumes`	private, `{user_id}/{session_id}.pdf`	App

Endpoint (POST /functions/v1/…)	Request	Response	Owner
parse-resume	`{ session_id }`	`{ resume_lines, jd_requirements }`	C
generate-questions	`{ session_id }`	`{ questions }` (6)	C
score-answer	`{ question_id, answer_text }`	Feedback + `session_completed`, `session_overall`	D

Errors: `{ error: { code, message } }`, code ∈ bad_request, unauthorized, not_found, llm_failed, parse_failed.

Stack: React + Vite + TypeScript + Tailwind (Antigravity); Supabase Auth, Postgres, Storage, 3 Deno Edge Functions; Gemini via free AI Studio key (Claude API optional fallback via `LLM_PROVIDER`); Vercel from GitHub `main`; `VITE_USE_MOCKS` switches fixtures vs real API.

6. Team, timeline (UK time) and success

Person	Owns	First deliverable
A (Pro)	Frontend `src/`, repo, contracts keeper	Whole flow clicks through on mocks
B	Supabase project, migration, auth, secrets	Signed-in user can insert a session + upload PDF, can't read others'
C	`llm.ts`, parse-resume, generate-questions	Curl with each fixture returns 6 questions citing real ids

Person	Owns	First deliverable
D	score-answer, fixtures, Vercel, QA	Weak answers score clearly below strong; invented claims flagged

Deadline **Sun 1 Nov 04:30 UK** (10:00 IST; UK is GMT since 25 Oct, 5 h 30 m behind IST).

When (UK)	Milestone
Thu 29 Oct evening	PRD + prompt done; B creates repo `aiap-c15-team44-roleready` + Supabase project, invites everyone
Fri 30 Oct 12:00	Contracts + schema frozen; fixtures + llm.ts ready
Fri 30 Oct 22:00	Frontend end to end on mocks; 3 functions deployed, curl tests pass
Sat 31 Oct 10:00	Integration: mocks off, flow works on Vercel
Sat 31 Oct 14:00	Must-haves done + bug bash; should-haves only after this
Sat 31 Oct 17:00	Feature freeze; Loom + deck from here
Sat 31 Oct 22:00	Submit (6 h 30 m buffer)

Success metrics: sign-in to first feedback < 3 min on the live URL; 6/6 questions cite a real R/J id; on fixtures the strong answer beats the weak by ≥ 25 points and every invented claim is flagged.